无线通信技术实验指导手册（学生版）

信道编码与信道均衡综合实验

基于 GitHub 的代码补全、自动评分与实验报告提交流程

课程：无线通信技术

实验学时：2 学时课堂实验 + 课后完善报告

仓库：<https://github.com/jwentong/wireless-coding-equalization-exp>

文档日期：2026-05-14

实验前准备（重要）

请同学们在实验课前尽量完成以下准备。课堂上教师会演示环境配置和 GitHub 提交流程，但提前准备可以节省实验时间。

项目	要求等级	说明
笔记本电脑	必须	建议携带充电器，保证能运行 VS Code 和 Python
GitHub 账号	必须	用于 Fork 仓库、提交 PR 和查看自动评分
VS Code	推荐提前安装	课堂会演示，也可提前从官网下载安装
Git	推荐提前安装	用于 clone、commit、push
Python	可课上配置	推荐 Python 3.9+，本实验依赖 NumPy、SciPy、Matplotlib、pytest
GitHub Copilot / AI 助手	可选	可辅助理解、补全和调试代

		码，但必须理解并验证生成代码

推荐提前安装：

```
git --version
python --version
```

如果命令不可用，课堂上可跟随教师或使用 Copilot Agent 辅助配置。

目录

一、实验概述

二、课堂时间安排

三、仓库结构与文件说明

四、实验环境配置

五、学生需要补全的代码

六、Part 1：Hamming(7,4) 信道编码

七、Part 2：ZF 与 LMS 信道均衡

八、本地运行与结果检查

九、实验报告要求

十、GitHub 提交与自动评分

十一、评分标准

十二、AI 助手使用边界

十三、常见问题

附录 A：命令速查

附录 B：PR 标题格式要求

一、实验概述

1.1 实验名称

信道编码与信道均衡综合实验。

1.2 实验背景

无线信道中常见的误码、噪声、多径传播和符号间干扰会降低通信系统可靠性。本实验围绕课程第 6 章“信道编码”和第 7 章“信道均衡”设计，通过代码补全方式完成两个核心模块：

- 用 Hamming(7,4) 线性分组码实现编码、伴随式检测和单比特纠错。
- 用 ZF 与 LMS 均衡器抑制多径信道带来的 ISI。

实验不是要求从零搭建工程，而是在已给定的实验平台中补全关键函数、运行仿真、生成结果图、提交 GitHub PR，并查看自动评分反馈。

1.3 实验目标

完成本实验后，你应能够：

- 理解信道编码通过冗余提升可靠性的思想。
- 掌握 Hamming(7,4) 生成矩阵、校验矩阵、伴随式和单比特纠错流程。
- 理解多径信道导致符号间干扰（ISI）的原因。
- 实现迫零（ZF）均衡器和 LMS 自适应均衡器。
- 能在本地运行 pytest 和评分脚本检查代码。
- 能通过 GitHub Pull Request 提交实验并查看自动评分结果。
- 能在实验报告中说明算法原理、结果图和 AI 辅助使用情况。

二、课堂时间安排

时间段	时长	内容	学生活动
0-15 分钟	15 min	实验背景、仓库结构、评分方式说明	了解本次实验要求
15-30 分钟	15 min	环境配置、Fork/Clone、运行环境检查	完成环境准备
30-45 分钟	15 min	Hamming(7,4) 编码与译码流程讲解	对照代码阅读 Part 1 TODO
45-60 分钟	15 min	ZF/LMS 均衡流程讲解	对照代码阅读 Part 2 TODO
60-100 分钟	40 min	学生独立补全核心代码	完成必做 TODO 并本地测试
100-115 分钟	15 min	结果图生成、报告模板说明	生成 results 中的图片
115-120 分钟	5 min	PR 提交和自动评分提醒	记录课后提交要求

如果课堂时间不足，优先完成 Part 1 和 Part 2 的必做函数；实验报告可课后完善。

三、仓库结构与文件说明

本实验仓库主要目录如下：

```
wireless-channel-coding-equalization-exp/
├── README.md                    # 学生快速指南
├── REPORT_TEMPLATE.md           # 实验报告模板
├── requirements.txt             # Python 依赖
├── src/                         # 学生主要补全代码的位置
│   ├── part1_channel_coding.py   # Part 1：信道编码 TODO
```

```
│   ├── part2_equalization.py     # Part 2：信道均衡 TODO
│   ├── utils.py                  # 已实现工具函数，不建议修改
│   └── test_environment.py       # 环境检查脚本
├── grading/                     # 自动评分脚本，不建议修改
│   ├── test_part1_coding.py
│   ├── test_part2_equalization.py
│   ├── check_report.py
│   └── calculate_grade.py
├── docs/                        # 理论文档和 Git 指南
├── results/                     # 结果图输出目录
└── .github/workflows/           # GitHub Actions 自动评分
```

学生主要修改以下文件：

- `src/part1_channel_coding.py`
- `src/part2_equalization.py`
- `REPORT.md`（由 `REPORT_TEMPLATE.md` 复制或新建）

除非教师要求，不要修改 `grading/` 中的测试和评分脚本。

四、实验环境配置

4.1 获取仓库

推荐流程：

1. 打开教师提供的 GitHub 仓库链接。
2. 点击 Fork 或 Use this template，生成自己的仓库副本。
3. 在本地终端运行：

```
git clone <你的仓库地址>
cd wireless-channel-coding-equalization-exp
```

4.2 安装依赖

在仓库根目录运行：

```
pip install -r requirements.txt
```

如果你使用虚拟环境，建议先创建并激活虚拟环境，再安装依赖。

4.3 检查环境

```
python src/test_environment.py
```

看到“环境配置正确”或类似通过提示，说明 NumPy、SciPy、Matplotlib、pytest 等依赖可用。

五、学生需要补全的代码

本实验采用“补全 TODO”的形式。打开 `src/` 目录中的两个 Python 文件，找到 `TODO` 和 `raise NotImplementedError`，用你的实现替换它们。

文件	函数	是否必做	主要任务
`src/part1_channel_coding.py`	`hamming74_encode(bits)`	必做	Hamming(7,4) 编码
`src/part1_channel_coding.py`	`hamming74_syndrome(codewords)`	必做	计算伴随式
`src/part1_channel_coding.py`	`hamming74_decode(received)`	必做	单比特纠错译码
`src/part1_channel_coding.py`	`convolutional_encode(bits)`	选做	卷积码编码
`src/part1_channel_coding.py`	`viterbi_decode_hard(received_bits)`	选做	Viterbi 硬判决译码
`src/part2_equalization.py`	`estimate_zf_equalizer(channel,	必	估计 ZF 均衡

	num_taps)`	做	器抽头
`src/part2_equalization.py`	`apply_fir_filter(signal, taps)`	必做	FIR 滤波输出
`src/part2_equalization.py`	`lms_equalizer(rx_train, tx_train, num_taps, step_size)`	必做	LMS 自适应均衡训练

补全代码时，请保留函数名、参数名和返回值格式，否则自动评分可能无法导入或调用你的函数。

六、Part 1：Hamming(7,4) 信道编码

6.1 理论提示

Hamming(7,4) 每 4 个信息比特生成 7 个码字比特，仓库中已给出生成矩阵 `HAMMING_G` 和校验矩阵 `HAMMING_H`。

- 编码：$c = uG \bmod 2$
- 伴随式：$s = rH^T \bmod 2$
- 若 $s = 0$，认为没有检测到单比特错误。
- 若 $s \neq 0$，将 s 与 `HAMMING_H` 的每一列比较，定位错误比特并翻转。
- 本实验中的系统码信息位为前 4 位。

6.2 函数 1：`hamming74_encode(bits)`

输入：一维 0/1 数组，长度必须是 4 的倍数。

输出：一维 0/1 编码比特数组，长度为输入长度的 $7/4$。

建议实现步骤：

4. 将 `bits` reshape 成形状 `(-1, 4)`。
5. 计算 `blocks @ HAMMING_G`。
6. 对结果取模 2。
7. 将二维码字数组 flatten 成一维数组返回。

本地检查示例：

```
python -m pytest grading/test_part1_coding.py -v
```

6.3 函数 2：`hamming74_syndrome(codewords)`

输入：一维或二维码字数组。若是一维，长度必须是 7 的倍数。

输出：形状为 `(N, 3)` 的伴随式数组。

建议实现步骤：

8. 如果输入是一维数组，reshape 为 `(-1, 7)`。
9. 计算 `codewords @ HAMMING_H.T`。
10. 对结果取模 2。
11. 返回伴随式矩阵。

6.4 函数 3：`hamming74_decode(received)`

输入：一维 0/1 接收序列，长度必须是 7 的倍数。

输出：纠错后的信息比特序列。

建议实现步骤：

12. 将 `received` reshape 为 `(-1, 7)`，建议复制一份避免直接修改输入。
13. 调用 `hamming74_syndrome` 计算每个码字的伴随式。
14. 对每个非零伴随式，与 `HAMMING_H` 的 7 列逐列比较。
15. 找到匹配列后，翻转对应码字位置。
16. 取每个码字前 4 位并 flatten 返回。

6.5 运行 Part 1

补全 Part 1 后运行：

```
python src/part1_channel_coding.py
```

成功后应生成：

`results/coding_ber_curve.png`

该图用于比较未编码和 Hamming(7,4) 编码后的 BER 表现。

七、Part 2：ZF 与 LMS 信道均衡

7.1 理论提示

多径信道会使接收符号包含相邻符号的叠加，产生符号间干扰（ISI）。均衡器的目标是设计一个 FIR 滤波器，使“信道 + 均衡器”的整体响应尽量接近一个冲激响应。

7.2 函数 4：`estimate_zf_equalizer(channel, num_taps)`

输入：信道冲激响应和均衡器抽头数。

输出：长度为 `num_taps` 的 ZF 均衡器系数。

建议实现步骤：

17. 构造卷积矩阵 `A`，使 `A @ taps` 表示 `channel` 与 `taps` 的卷积结果。
18. 构造目标冲激响应 `d`，在中心位置放置 1，其余位置为 0。
19. 使用 `np.linalg.lstsq(A, d, rcond=None)` 求最小二乘解。
20. 返回 `taps`。

检查重点：

- 返回长度必须等于 `num_taps`。
- `np.convolve(channel, taps)` 的主峰应明显大于旁瓣能量。

7.3 函数 5：`apply_fir_filter(signal, taps)`

输入：一维信号和 FIR 抽头。

输出：与输入 `signal` 等长的滤波结果。

建议实现步骤：

21. 使用 `np.convolve(signal, taps, mode='full')`。
22. 截取前 `len(signal)` 个样本。
23. 返回截取后的数组。

自动测试会检查单位抽头 `[1.0]` 是否保持信号不变，以及结果是否与 `np.convolve(..., mode='full')[:len(signal)]` 一致。

7.4 函数 6：`lms_equalizer(rx_train, tx_train, num_taps, step_size)`

输入：接收训练序列、期望发送训练符号、抽头数、步长。

输出：训练后的抽头 `taps` 和误差序列 `errors`。

建议实现步骤：

24. 初始化 `taps`，可令中心抽头为 1，其余为 0。
25. 从第 `num_taps - 1` 个样本开始迭代。
26. 构造当前输入向量 `x`，长度为 `num_taps`。
27. 计算输出 `y = taps @ x`。
28. 计算误差 `e = d - y`，其中 `d` 来自 `tx_train`。
29. 根据 LMS 公式更新：`taps = taps + step_size * e * x`。
30. 保存每次迭代的误差并返回。

调试建议：

- 如果误差不下降，检查输入向量是否与期望符号对齐。
- 如果结果发散，尝试减小 `step_size`。
- 如果数组长度不一致，优先检查循环起点和 `x` 的切片方向。

7.5 运行 Part 2

补全 Part 2 后运行：

```
python src/part2_equalization.py
```

成功后应生成：

```
results/equalization_eye_comparison.png
results/equalization_mse_curve.png
```

八、本地运行与结果检查

建议按以下顺序检查：

```
python src/test_environment.py
python -m pytest grading/test_part1_coding.py -v
python -m pytest grading/test_part2_equalization.py -v
python src/part1_channel_coding.py
python src/part2_equalization.py
python grading/calculate_grade.py
```

结果目录至少应包含：

```
results/coding_ber_curve.png
results/equalization_eye_comparison.png
results/equalization_mse_curve.png
```

注意：pytest 中结果图检查是宽容的。如果图片尚未生成，会显示 skipped；但最终提交前请务必运行两个实验脚本生成图片。

九、实验报告要求（报告提交至 eejwentong@szu.edu.cn）

请根据 `REPORT_TEMPLATE.md` 编写 `REPORT.md`，至少包含：

31. 实验目的。
32. Hamming(7,4)、伴随式、单比特纠错原理。
33. ISI、ZF、LMS 均衡原理。
34. 实验环境和主要依赖。
35. Part 1 和 Part 2 的实验步骤。

36. 三张结果图及说明。

37. 结果分析与讨论。

38. AI 助手使用情况说明。

39. 参考资料。

结果图引用示例：

```
![编码 BER 曲线](results/coding_ber_curve.png)
![均衡眼图对比](results/equalization_eye_comparison.png)
![LMS 误差曲线](results/equalization_mse_curve.png)
```

十、GitHub 提交与自动评分

10.1 提交代码

确认本地测试通过后运行：

```
git status
git add src/part1_channel_coding.py src/part2_equalization.py REPORT.md
results/
git commit -m "Complete channel coding and equalization experiment"
git push origin main
```

10.2 创建 Pull Request

在 GitHub 页面创建 Pull Request 到教师仓库。为了方便教师跟踪作业，请使用以下 PR 标题格式：

实验 `02-姓名-学号`

示例：

实验 `02-张桂嘉-2022040399`

如果还没有登记 GitHub 用户名，系统会尝试从 PR 标题中的姓名或学号匹配到学生名单。

10.3 查看自动评分

创建 PR 后等待 3-5 分钟，评分结果可能出现在三个位置：

40. PR 页面下方的自动评分评论。
41. GitHub Actions 的 Summary。
42. GitHub Actions 的 Artifacts，可下载 `grade_report.json` 和日志文件。

如果 PR 评论没有出现，不代表没有评分。请先打开 Actions Summary 和 Artifacts 查看结果。

10.4 重新触发评分

如果你修改了代码：

```
git add .
git commit -m "Fix coding and equalization implementation"
git push origin main
```

PR 会自动重新评分。不要只点击旧 workflow 的 Re-run，因为旧 workflow 可能使用旧配置。

十一、评分标准

项目	分值	自动评分内容
环境配置	5	`src/test_environment.py` 通过
Part 1：Hamming(7,4) 信道编码	35	编码、伴随式、单比特纠错、BER 结果图
Part 2：信道均衡	35	FIR 滤波、ZF 均衡、LMS 训练、均衡结果图
实验报告	15	章节完整、结果图、分析讨论、AI 使用说明
代码质量	-10~+5	pylint 评分，优秀加分，较差扣分

选做加分	+10	卷积码/Viterbi 或其他经教师认可的扩展

最终总分限制在 0~100 分。自动评分主要检查代码正确性和报告完整性，教师仍可根据课堂表现、报告分析深度和学术诚信进行复核。

十二、AI 助手使用边界

可以使用 Copilot、ChatGPT 或其他 AI 助手辅助理解和调试，但必须遵守：

- 可以让 AI 解释 Hamming(7,4)、ZF、LMS 的原理。
- 可以让 AI 帮助分析报错、补全思路或生成测试用例。
- 不要直接提交自己不理解、未运行、未验证的代码。
- 不要修改评分脚本来规避测试。
- 必须在 `REPORT.md` 中说明 AI 辅助使用情况。

推荐提问模板：

请解释 `Hamming(7,4)` 中 `syndrome` 如何定位单比特错误，结合当前 `src/part1_channel_coding.py` 的 `HAMMING_H` 说明。

我的 `lms_equalizer` 误差不下降，请根据 `LMS` 公式检查输入向量、期望符号和更新方向可能哪里错了。

十三、常见问题

Q1：运行时报 `ModuleNotFoundError` 怎么办？

在仓库根目录运行：

```
pip install -r requirements.txt
```

如果你使用虚拟环境，请确认 VS Code 当前选择的是同一个 Python 环境。

Q2：`NotImplementedError` 是什么意思？

说明某个 TODO 函数还没有补全。打开报错提示对应的文件和函数，用你的实现替换 `raise NotImplementedError(...)`。

Q3：Part 1 结果图没有生成怎么办？

先确认三个必做函数都已实现，再运行：

```
python src/part1_channel_coding.py
```

如果仍失败，查看终端中的具体异常信息。

Q4：LMS 误差曲线不下降怎么办？

检查以下几点：

- `rx_train` 和 `tx_train` 是否长度一致。
- `x` 的长度是否等于 `num_taps`。
- `y = taps @ x` 的方向是否正确。
- 更新公式是否为 `taps = taps + step_size * e * x`。
- `step_size` 是否过大导致发散。

Q5：GitHub Actions 没有 PR 评论怎么办？

评论权限可能受 fork 或组织策略影响。请查看 Actions Summary 和 Artifacts。评分脚本会尽量把结果写到这两个位置。

Q6：可以多次提交吗？

可以。每次 push 新 commit 后，PR 会自动重新评分。建议每次提交前先在本地运行测试，避免无效提交。

Q7：我没有填写 GitHub 用户名，教师还能跟踪我的提交吗？

可以，但 PR 标题必须包含姓名或学号。推荐格式：`实验 02-姓名-学号`。

Q8：可以修改 `utils.py` 或 `grading/` 吗？

一般不建议。学生应主要修改 `src/part1_channel_coding.py`、`src/part2_equalization.py` 和 `REPORT.md`。修改评分脚本不会提高 GitHub 自动评分，且可能被视为无效提交。

附录 A：命令速查

命令	说明
`pip install -r requirements.txt`	安装实验依赖
`python src/test_environment.py`	检查 Python 环境
`python -m pytest grading/test_part1_coding.py -v`	运行 Part 1 测试
`python -m pytest grading/test_part2_equalization.py -v`	运行 Part 2 测试
`python src/part1_channel_coding.py`	生成编码 BER 曲线
`python src/part2_equalization.py`	生成均衡结果图
`python grading/calculate_grade.py`	本地计算总分
`git status`	查看文件修改状态
`git add .`	添加全部修改
`git commit -m "message"`	提交到本地仓库
`git push origin main`	推送到个人 GitHub 仓库

附录 B：提交检查清单

提交 PR 前，请逐项确认：

- 已补全 Part 1 的 3 个必做函数。
- 已补全 Part 2 的 3 个必做函数。
- 已运行两个实验脚本并生成 3 张结果图。
- 已根据模板完成 `REPORT.md`。
- 已运行 `python grading/calculate_grade.py` 查看本地评分。
- PR 标题包含姓名和学号。

- 没有提交无关临时文件，例如 Word 的 `~$` 文件。